

# Documento de Testing

## Growshop Sativamente

**Nombre del proyecto:** Growshop Sativamente.

**Integrantes:** Jean Carlos Flores. Joshua Mardones.

**Sección:** Ingeniería en Informática, Full Stack II, 006D

**Docente:** Viviana Poblete **Fecha:** 25 de noviembre de 2025

---

## Introducción

El presente documento resume las pruebas realizadas al sistema web **Growshop Sativamente**, enfocadas en verificar el correcto funcionamiento del catálogo de productos, el carrito de compras, la navegación del sitio y el módulo de administración. Estas pruebas se desarrollan en el contexto de la **Experiencia 3 de Full Stack II**, con el propósito de asegurar que las funciones principales del frontend operen de forma consistente, accesible y alineada a los requerimientos definidos para la plataforma.

## Objetivo del Documento

El objetivo de este documento es **describir y registrar las pruebas ejecutadas** sobre los componentes clave del sistema Sativamente, validando:

- El comportamiento correcto del catálogo, carrito y Navbar.
- La autenticación y gestión de productos dentro del módulo Admin.
- La respuesta del sistema ante interacciones reales del usuario.

Este documento entrega evidencia del funcionamiento del sistema y respalda su evaluación dentro del proyecto académico.

# 1. Herramientas Utilizadas en el Testing

El proceso de pruebas del proyecto Sativamente se realizó utilizando un conjunto de librerías orientadas a validar el comportamiento del frontend y los flujos principales del sistema.

## **1.1 Vitest**

Framework principal encargado de ejecutar las pruebas unitarias.

### **Funciones usadas:**

- describe
- it
- expect
- beforeEach
- vi

### **Utilidad en el proyecto Sativamente:**

- Organización de suites de pruebas para componentes como Catálogo, Navbar y Admin.
- Control del entorno con mocks.
- Ejecución rápida y aislada sin servidor backend real.

## **1.2 React Testing Library**

Librería centrada en probar componentes desde la perspectiva del usuario.

### **Funciones utilizadas:**

- render
- screen
- waitFor

### **Uso específico:**

- Renderizado del catálogo de productos de cultivo.
- Verificación del contenido del carrito.
- Evaluación de botones (Agregar al carrito, Ver más).

## **1.3 @testing-library/user-event**

Permite simular interacciones reales del usuario.

**Acciones probadas:**

- Clic en productos del catálogo.
- Escritura en formularios de inicio de sesión del administrador.
- Interacción con los botones del panel administrativo.

#### **1.4 jest-dom**

Agrega aserciones específicas del DOM para mayor precisión.

**Validaciones aplicadas:**

- `toBeInTheDocument()`
- `toBeVisible()`
- `toHaveClass()`

Usadas principalmente para confirmar la visibilidad de tarjetas de productos, alertas y mensajes del sistema.

#### **1.5 React Router – MemoryRouter**

Se utilizó para simular navegación al probar:

- Navbar
- Botones del catálogo que dirigen al detalle del producto
- Componente Admin con rutas protegidas

#### **1.6 Simulación de funciones globales**

Se mockearon funciones del navegador para evitar interacciones reales:

- `alert`
- `confirm`
- `localStorage` (fundamental para el carrito)

## **2. Alcance del Testing**

El testing se enfocó en validar el comportamiento de las secciones críticas del sistema: catálogo, carrito y administración.

## **2.1 Pruebas del Catálogo de Productos**

Se evaluó:

- Renderizado correcto de productos de cultivo (fertilizantes, sustratos, iluminación).
- Comportamiento del botón **“Agregar al carrito”**.
- Visualización de precios y stock.
- Funcionamiento del botón **“Ver Detalle”**.

## **2.2 Pruebas del Carrito**

Incluyen:

- Almacenar información del carrito en localStorage.
- Actualización de cantidades.
- Eliminación de productos.
- Visualización del total.

## **2.3 Pruebas del Administrador**

Se probaron funcionalidades exclusivas del administrador:

- Inicio de sesión con credenciales mockeadas.
- Carga de productos mediante API simulada.
- Activar/desactivar productos.
- Modificar y eliminar productos.
- Visualización de tabla con categorías, precios y estado.

## **2.4 Pruebas del Navbar**

Incluyen:

- Enlaces principales del sitio (Inicio, Catálogo, Carrito).
- Visualización dinámica del contador del carrito.
- Comportamiento al cambiar de sección mediante MemoryRouter.

# **3. Detalle de Pruebas Realizadas**

### **3.1 Autenticación del Administrador**

Casos cubiertos:

- Mostrar formulario de acceso al cargar el admin.
- Validar ingreso con contraseña correcta.
- Error en contraseña incorrecta → mostrar alerta.

### **3.2 Renderizado del Catálogo de Sativamente**

Se valida:

- Listado de productos provenientes del mock.
- Correcto display de categorías y precios.
- Botón **Agregar al carrito** presente en cada producto.

### **3.3 Agregar productos al carrito**

Casos:

- Agregar un producto → actualizar visualmente.
- Repetir acción → aumentar cantidad.
- Guardar todo en localStorage.

### **3.4 Vista y edición del panel administrativo**

- Ver listado completo de productos.
- Cambiar el estado de un producto (ej. activar/desactivar un fertilizante).
- Eliminar producto con confirmación.
- Editar campos como precio, categoría o nombre.

### **3.5 Formularios de creación y edición de productos**

Pruebas:

- Render del formulario al seleccionar “Nuevo Producto”.
- Validación de campos vacíos.
- Envío correcto del formulario al completar los datos.

### **3.6 Accesibilidad mínima**

- Roles correctos en botones.
- Etiquetas visibles en formularios.
- Lectura correcta del nombre de productos.

### **3.7 Navbar y Contador del Carrito**

- Se renderizan correctamente los enlaces Inicio, Catálogo y Carrito.
- El ícono del carrito muestra el número correcto de productos.
- Contador se actualiza tras usar localStorage.

## **4. Configuración del Entorno de Pruebas**

### **Se utilizó un archivo setupTests.js que contiene:**

```
import '@testing-library/jest-dom'
```

Esto habilita aserciones adicionales del DOM en todos los tests del proyecto.

### **Se configuraron también:**

- Mocks para localStorage
- Mocks de las APIs del Backend
- Simulación de rutas con MemoryRouter

## **5. Conclusiones**

El proceso de testing realizado para **Growshop Sativamente** permitió validar las funcionalidades esenciales del sistema, asegurando que los componentes que sostienen la experiencia del usuario operan de manera confiable y consistente.

El uso de Vitest junto con React Testing Library hizo posible probar interacciones reales, especialmente en áreas sensibles como el carrito de compras y la administración del catálogo de cultivo. La simulación de APIs y funciones globales permitió realizar pruebas aisladas, evitando dependencias externas y logrando resultados reproducibles.

Gracias a estas pruebas, el equipo pudo confirmar que el sistema responde adecuadamente a las acciones del usuario, mantiene la integridad de los datos y cumple con los flujos requeridos por la experiencia 3, alineándose tanto al contexto académico como a los objetivos comerciales de la tienda Sativamente.